Pandas

Pandas is a library of python programming language.

→ It is mostly used for working on datasets.

Utility of Pandas:- I. Data Manipulation → Inserting, modifying, removing, deleting etc. existing data (i.e. special, mean, median, deleting etc.)

II. Data Analysis → analyzing the data by statistical & Mathematical formulas, etc. (mean)

III. Data Visualization → [raw data printing is known as visualization i.e. bar, pie chart etc.]

Manipulation:- i.e; applying CRUD operations on given data.

C (Create) → Creating some new rows

R (read) → Reading already existing rows

U (Updating) → Updating the already existing rows

D (deletion) → Deleting already existing rows

→ Matplotlib > Only for visualization using various plots.
Seaborn

→ Pandas handles tabular data → excel, csv, xlsx.

Importing Pandas & Installing.

Import pandas as pd !Pip install pandas

(or).

Import pandas as —
- pd → alias
- that is used to name it.

[It is a pandas Data Structure].

Data Frame → It is an Object Contain multiple rows & columns.
Similar to numpy 2dim array.

Collection of series is dataframe

Data structure in Pandas : Data frame

Series

In tabular format

Rows → Entities / Data Points.

Columns → Properties of Entities / Data points

(or)

Attributes

(or)

Features

name state age high school → Properties of entity

→ entity

→ Each columns are unique so it is heterogeneous

So in numpy only homogeneous allowed whereas in pandas it heterogeneous respective to column.

→ To read tabular data

df = pd.read_csv(Path)

dataframe. filename relative / absolute path.

→ Saving the file

One manipulation is done we have to save the file to hard drive again.

ie; df.to_csv(Path).

dataframe where Path to save the df.

to Save

CSV format

Series is Homogeneous (ie; it is a Column)

DataFrame is heterogeneous.

→ df.head() → by default 5 will take → head
df.tail() → bottom 5 rows 5 will take → tail
df.dtypes()
df.info() → to check the datatypes of each column
df.describe() → mean, std, min ... max → statistics
dtype → by attribute

→ Series [similar to 1D Numpy Array] — size & immutable
↓ Homogeneous ↓ Homogeneous data type

→ Dataframe (2D numpy array) — size is mutable (add = row or column)
↓ Heterogeneous ↓ Homogeneous

Series :- Syntax :- s = pd.Series(data, Index)
(or)
s = pd.Series(data)

→ Difference in Series "S" is Capital
→ Data should be a python list, tuple or dict, or ndarray (numpy array), a Scalar value (like 10)
→ Index — It is optional if we don't give any data it takes default index as 0 to n-1.
→ In pandas string dtype is called as an Object.

Python list & dict

eg:- a = pd.Series([1,2,3])
Print(a) → index of row.
O/p:- 0 1
1 2
2 3
dtype: Int64. → 8×8 = 64

eg:- a = pd.Series([1,2,3], index = ['a','b','c']).
Print(a).
O/p:- a 1
b 2
c 3
dtype: Int64

→ Numpy Array

→ Scalars

→ To Series data must be in 1 dimensional. If we give (10, 20, 30)

o/p: error

Data must be 1-dim.

Accessing Properties/Attributes and Methods of Series.

1. dtype — what is the data type in Series
2. values — values in the pandas Array.
3. array — It shows the data, type of the array. dtype [How many elements].
5. shape — shape(3,) → (3, 2) ex: [10, 20, 30, 40, 50, 60]

5. to_numpy() → Extracts the actual ndarray from Series.

↓

It is a method of Series.

For method → we have ( ).

For Features/Properties → No parenthesis.

→ s.dtypes → what type the data type in dataframe.
→ s.values() → dtype, numpy array

Indexing and Slicing:-

Indexing and slicing in pandas Series are straightforward and similar to how you would index and slice lists or NumPy arrays.

i. Accessing a single element by Position

```
eg:- a = [10, 20, 30, 40]
     s = pd.Series(a)
     print(s[2])   o/p:- 30.
```

ii. Accessing a single element by label.

```
eg:- s = pd.Series(a, index=['a','b','c','d'])
     Print(s['c'])   o/p:- 30.
```

→ Slicing by Position.

```
eg:- Print(s[1:3])   # slicing from 1 to 3 position (exclusive of 3 position)
     o/p:- b  20
           c  30.
           dtype: Int64.
```

→ Slicing by label

```
eg:- print(s['b':'d'])   # slicing from 'b' to 'd' (inclusive 'd').
     o/p:- b  20
           c  30
           d  40.
```

getPrint(s['f']) o/p:- KeyError. [If is not label name in index]

→ get() — 'get' method in series is used to retrieve a value at a specified label (index). If the label does not exist in the series, it can return a default value.

```
eg:- Print(s.get('f')).   o/p:- None
     Print(s.get('f', nanan)).   o/p:- nan
```

DataFrame

Pandas Data structures are value mutable (the values they contain can be altered) as well as size mutable.

→ When working with tabular data, such as data stored in spreadsheets or databases, pandas is the right tool for you. pandas will help you to explore, clean, process your data. In pandas, a data table is called a DataFrame.

→ A table of data is stored as a Pandas DataFrame.

Each column in a DataFrame is a Series.

For creating DF in Pandas:-

Syntax:- df = pd.DataFrame(data, index=index, columns=cols)

data can be in different forms. i.e., Python Dict, List or type Numpy Array

eg:- Using Python dictionary.

```
data = { 'name': ['ge', 'vi'],
         'age': [23, 25],
         'Gender': ['F', 'M'] }
df = pd.DataFrame(data)
df
```

o/p:-

	name	age	Gender
0	ge	23	F.
1	vi	25	M.

Print (type(df['name'])) → o/p:- Series.

Print (type(df[['name']]) → o/p:- DataFrame.

→ keys will be interpreted as column names.

→ Values mapped to keys will be data inside dataFrame.

→ Series is nothing but a Col.

→ For accessing the column in a dataframe

df[' '] → type of this Series

↓ Column name

→ For accessing Multiple Columns

df[['cols', 'cols']] → type of dataframe

df[['cols']] → bracket inside bracket is dataframe.

Accessing Attributes/Properties and Methods of DF:-

df.shape — Gives the tuple representing the dimensionality of Dataframe

o/p:- (5, 3)
rows cols

df.columns:- Returns the column labels of the Dataframe.

o/p:- Index['name', 'age'], dtype='object')

df.dtypes:- Returns the data types of each column

o/p:- name : object
age int64
dtype: object

df.axes:- What are name of rows & cols & dtype.

df.values:- Returns the numpy array for dataframe

df.index:- Returns the row labels of Dataframe.

Methods:-

df.info():- Provides a concise summary of the Dataframe.

i. datatype
ii. Total no of rows.
iii. Cols.
iv. Name of cols
v. data types
vi. Memory Usage → in RAM.

df.head():- by default head returns first 5 rows.

df.head(2) → gives 2 rows.

df.tail():- by default tail returns last 5 rows.

df.tail(6) → bottom 6 rows.

Working with Tabular Data:-

3 components in a Computer
- CPU - faster
- RAM - Memory (variables, Stored)
- HDD (files, variables)

.csv, .xlsx → files in HDD

→ df.to_excel("Sheet1.xlsx")

→ for actual path

df.to_csv(path)

df.to_csv(path, index=False)
↳ No index.

csv — It is a workbook - collection of sheets.

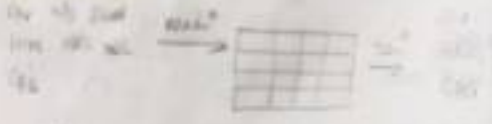
→ Reading .xlsx file

excel

df = pd.read_excel(" .xlsx")

for csv, = pd.read_csv(" .csv").

→ Pandas supports many file formats or data sources out of the box (csv, excel, sql, json, parquet ...). For importing data from each of these data sources is provided by function with prefix "read_", "to_" methods are used to store data.



→ Dataframe to csv.

df.to_csv("path to save".csv")

→ without index.

df.to_csv("path to save .csv", index=False).

→ Dataframe to xlsx.

df.to_excel("path to save in folder etc .xlsx", sheet_name='Sheet').

without index

index=False)

Non-Visual Data Analysis using Pandas (Statistical Analysis).

→ Aggregation Statistics (mean, median, min, max, count) can be calculated on entire Columns or rows

→ Groupby provides the power of the split-apply-combine pattern

→ value_counts is a convenient shortcut to count the unique values in each category of a variable.

→ sum( ):- Returns the sum of values for requested axis. by default axis=0.

df.sum().

axis=1 → rowwise sum.

df[['cats','dogs']].sum(axis) → for selected Columns.

→ min( ):- The min( ) method returns the minimum value for each column by default.

max( ):- The max( ) " " " maximum " " "

axis=1 → for rowwise min/max value.

→ mean( ):- The 'mean( )' method returns the mean (average) value for each Column by default.

for each row by specifying 'axis=1'

df.mean(axis=1)

→ Median( ):- The 'median( )' method returns the median value for each col by default, for each row by specifying 'axis=1'.

df.median(axis=1)

→ Variance( ):- The 'var( )' method returns the variance for each Column by default, for each row by specifying 'axis=1'.

df.var( ).

→ Standard Deviation:- The 'std( )' method returns the standard deviation for each column by default, axis=1 → for row std.

df.std().

df.sum(axis=1).

↓

we get Warning - We get the output (doesn't give abnormal termination of code)

shows Error - total Error

→ Because we categorical data types to save the given memory
→ We use tag to select the columns which are numeric

Syntax: DataFrame.select_dtypes(include=None, exclude=None)
DataFrame.select_dtypes() is a method in pandas to select columns in a DataFrame based on column dtypes.

Include: scalar or list-like

A data type or a list of data types to include in the returned DataFrame.
i.e. int, float, bool, datetime etc.

exclude: scalar or list-like

A data type or a list of data types to exclude from the returned DataFrame. Possible values are the same as those for 'include'.

→ If both 'include' & 'exclude' are done the method returns the original DataFrame. It handles both single & list of data types.

→ Code to ignore the Warnings.

```
import warnings
warnings.filterwarnings('ignore')
```

→ count( ):- It gives the no of rows in column. eg- df['species'].count()

→ unique( ):- It gives the unique categories inside the data, what are the categories

→ value_counts( ):- This method a series containing counts of unique values in descending order.

```
eg:- A 4
     B 3
```

→ nunique( ):- It gives the no of unique categories inside the data, how many categories.

```
eg:- 3.
```

[How many unique characters.]

→ KB — kilo Bytes.

Kb — kilo bits.

describe( ):- Summarizing the data
It gives the values in total column format.
Count — No. of values in each column (total count)
mean — Average
50% — Median — 50 percentile (0.50)
75% — 75th percentile — 75 percentile (0.75)
25% — 25th percentile
min
max — min & max values in column

→ describe( ): for non Numerical Columns use
Count 150
Unique 3
top Iris-setosa
freq 50

df.describe(include=[np.number]) → for numerical.
df.describe(include=['object']) → for object.
df.describe(include='all') → for all data type description.

→ Corr( ):- This method calculates the pairwise correlation of columns, excluding NA/null values.

df.corr( )
integers float object.

→ skew( ):- Skewness of the distribution of a column.
df['numerical col'].skew( )

→ kurt( ):- gives the kurtosis of the distribution of a column.
df['numerical col'].kurt( ).

DataFrame.agg() :- Instead of the predefined statistics, specific combination of aggregating statistics for given columns can be defined using the DataFrame.agg() method.

df.agg({f, l})

additional conditions we have to mention :- in the dict format.

eg:- df.agg({'species':['min', 'max'], 'sepallength(cm)':['min', 'max', 'count']})

Accessing Data in a DataFrame using Indexing & Slicing in Pandas :-

→ When selecting subsets of data, square brackets [ ] are used. Inside these brackets, you can use a single column/row label, a list of column/row labels, a slice of labels, a conditional expression or a colon.

accessing:

df['colname'] → for single column, Series datatype

df[['cols']] → for single/multiple columns, but it is in dataframe format.

eg:- df[['cols', 'cols']]

Filtering Rows from a DataFrame :-

Ways :- We can select the rows by using slicing operation.

Syntax :- df[starting row index : ending row index : step]

Ways :- Similar to numpy, pandas can accept boolean indexes.

To select rows based on a conditional expression, use a condition inside the selection brackets []

Syntax :- df[Condition]. → filter rows based upon a condition or multiple conditions

df[1] → 1st row should be output but we get the error.

→ Rows filtered based upon the condition.

eg: df['sepallength(cm)'] > 3.5.

ooo[F, F, T]. Here it gives boolean values True or False.

→ df[df['Sepallength'] > 5.2]

O/p:- It gives the dataframe > 5.2 Sepallength

It will gives the element.

→ Conditional expressions are — → bool dtype = [True, False, ...]

isin() :- It is a conditional function, returns a True for each row the values are in the provided list ([ ])

| (or) Operator:- to filtering by rows.

df[df['date'].isin(['10-5-2016'])]

O/p:- total dataframe where date is 10-5-2016.

df[df['date'] == '10-5-2016' | df['date'] == '10-4-2016']

O/p:- These date rows.

Note:- When combining multiple conditional statements, each condition must be surrounded by parentheses ( ). Moreover, you can not use or/and but need to use the or operator | and the and operator &.

Boolean or - '|'

Boolean and - '&'.

df.loc[ ]. → label based accessing.

Syntax:- df.loc[row_label, col_label]

Eg:- df.loc[100] → 100th row & all col values

df.loc[100:200, ['day', 'temp']].

↓

100th row to 200th row of day & temp cols.

df.loc[1:0:2, ['day', 'event']].

↓

Step 2.

	day	event
1	1/2/2016	Snow
3	3/4/2016	[illegible]
5	5/6/2016	Snow

df.iloc[ ]. → index based accessing.

Eg: df.iloc[row_index, col_index]

Eg:- df.iloc[100] → 100th row & all col values

df.iloc[100:200, [0, 1]] → 100 to 200th row of 0th index & 1st index

Columns (starts from 0)

row labels (starts from 0)

df.iloc[1:5:2, [2, 3]] → 1st to 5th Row step 2 of column index 2 & 3

column index is from 0 (zero)

df['column'] → returns a single column in series type

df[['column']] → returns a single column in dataframe type

df[['col1', 'col2']] → returns multiple cols in dataframe type

Accessing rows based on a condition in loc:-

df.loc[condition, col/labels]

eg: df.loc[df['max'] > 90, 'date']

↓

for single column

for multiple columns we give ['cols', 'cols']

Accessing rows based on multiple condition in loc:-

df.loc[(Cond1) & (Cond2) | (Cond3), Col/Labels].

eg: df.loc[(df['max'] > 90) & (df['low'] > 80), 'date']

Renaming Columns:-

→ How to create new columns derived from existing columns:-

Data Manipulations on a column work elementwise. Adding a column is a Dataframe based on other columns is straightforward.

rename() with a dictionary or function to rename row labels or column names

Syntax: df.rename(index=None, columns=None)

df1 = df.rename(columns={'DES': 'prod', 'low': 'cost'})

df1 # Replace Des we get prod, replace of low we get cost as column names.

df.strip():- to remove the extra spaces

df.lower():- Convert to lower cases

Remove all the special characters including spaces

→ Benefit of this is, we can now access the columns in the dataframe using the dot, similar to how we access the properties/attributes of a python object. For eg:- df1.cost → 1000.

→ Here — we can use this.

Modifying Columns Datatype:-

Modifying the Datatype using DataFrame.astype().

We can pass any python, Numpy or Pandas datatype to change all columns of a DataFrame to that type, or we can pass a dictionary having column names as keys and datatypes as values to change the types of specified columns.

eg:- df = df.astype(int)

eg: df = df.astype({'quantity': int, 'sunday': str})

df.dtypes

Modifying the Datatype using DataFrame.apply()

We can pass pandas.to_numeric, pandas.to_datetime, and pandas.to_timedelta as arguments to apply the apply() function to change the data types of one or more columns to numeric, DateTime, and time delta respectively.

eg:- df = df[['invoice date']].apply(pd.to_datetime);

→ When we apply any operation it will apply to all the elements.

Creating a derived Column.

→ Creating a new Column by merging other cols

eg:- df[Amount] = df[Qu.]* df[' Unit'].

df.head()

→ Calculation is element-wise, so the '+' is applied to the values in each row.

(+, -, /, *, ...) or logical Operators (<, >, ==, ...) work element-wise.

Creating a Column using apply() function.

Syntax:- df.apply(function, axis=0).

Applies the function Column-wise.

Axis:- Axis can be [0 or 'index', 1 or 'Columns'], default 0:

0 or 'index': apply function to each Column.

1 or 'Columns': apply function to each row.

Syntax for Series - axis.apply (function, axis=0)

Applies function elementwise

g= df.apply(square) ——→ by default axis=0.

→ df['square'] = df['quantity'] * df['unit price'].

by using apply function

g= df['new_column'] = df.apply (lambda row: row['quantity'] * row['unit price'], axis=1 for row wise)

axis=1 is compulsory to dealing columns. Otherwise it returns error.

Deleting Columns in Dataframe:-

Syntax 1:- Dropping columns by using column name.

→ Dropping columns by passing column names.

→ inplace = True, the changes are saved to the dataframe.

→ Without inplace = True → it will show just what it looks after delete. It's a view.

df.drop(['cols', 'cols'], axis=1, inplace=True)

Syntax 2:- Removing columns by using columns name using loc[].

Removing all columns b/w cols to cols.

df.drop(df.loc[:, 'cols':'cols'], inplace=True, axis=1)

g= df.drop(df.loc[:, 'smart':'noise'], inplace=True, axis=1)

Syntax 3:- Removing column based on index.

Removing columns by giving column indexes.

df.drop(df.columns[[0,3,4]], inplace=True, axis=1)

drops 0,3,4 index columns.

Syntax 4:- Removing column based on index using iloc[].

removing the columns between index 1 to 3.

df.drop(df.loc[:, [1,3]], inplace=True, axis=1) --> drop columns 1 & 3
[1,3] columns

Syntax 5:- DataFrame.pop() method.

It can delete single column at a time.

[pdvar = df.pop('col2')]

→ Removes the single column and returns the deleted column

→ It applies the changes to the dataframe without any need of inplace=True

Insert Rows) using Dictionary - pandas.concat()

Syntax 1:- Inserting a single row.

Creating a new record using dictionary

new_records = pd.DataFrame([{'day': 'friday', 'temperature': 35, 'city': 4, 'rain': 36}])

Inserting row at the end.

df = pd.concat([df, new_records], ignore_index=True)

Inserting row at the top

df = pd.concat([new_records, df], ignore_index=True)

→ By ignore_index=True

without ignore_index=True		with ignore_index=True
	0	0
	1	1
	2	2
	3	3
	0	4
	1	5

Syntax 2:- Insert multiple rows (ie; a batch of data)

Create a new record of dictionary

batch = pd.DataFrame([{'day': 'friday', 'temp': 'sunny'}, { }, { }])

Inserting row at the end.

df = pd.concat([df, batch], ignore_index=True)

Inserting row at the top

df = pd.concat([batch, df], ignore_index=True)

Inserting a Row using List - .loc[] and .iloc[]

To add a list to a pandas DataFrame using a list, alternatively, when we want simply use the .append() function. In order to do this, we need to use the loc accessor. The label that we use for our loc accessor will be the length of the DataFrame. This will create a new row.

Eg 1:- df.loc[len(df)] = [9.2, 200, 20.2, 'Rain'].

Eg 2:- df.iloc[] → Generates an Error - You cannot use iloc to enlarge the target object (i.e; iloc can't be used to add new rows)

df.loc[len(df.shape[0])] = [9.2, 200, 20.2, 'Rain'].

Inserting a Row at a Specific Index of a DataFrame

If we have current indices from 0-9 and we want to insert a new row at index 9, we can simply assign it using index 9.5. Lets see this work.

Adding a new label #6

df.loc[9.5] = [9.2, 200, 23.3, 'Rain'].

Sort index.

df = df.sort_index(), reset_index(drop=True)

df.

Handling Time Series Data:-

→ Valid date strings can be converted to datetime objects using to_datetime function

→ Pandas Datetime objects in pandas support calculations, logical operations and convenient date-related properties using the dt accessor like year, month, day, day_of_week, day_of_year, is_leap_year, week etc.

→ day_name(), month_name() etc.

→ pandas Timedelta represents a duration, the difference two dates or times. Many properties of timedelta can be use. Can also access timedelta like total_seconds().

pd.to_datetime( )

df[['date', 'date']].apply(pd.to_datetime) → to get many warnings that I don't know how to handle there.

These warnings are generated for a reason. Since dates can be specified in various formats, for ex: 20/01/2022 or 2022/01/20 or mm/dd/yyyy etc.

→ This warning is to specify the format (at how dates are stored in the date time column) so that you can prevent any parsing errors in future.

There are two ways to get rid of these warnings:

Way 1 :- Add parameter dayfirst=True.

Way 2 :- Add " → format='%d/%m/%y'.

Timedelta:- It is nothing but a difference b/w two dates or times.

Ex:- pd.timedelta(df['ship date'], dayfirst=True)

Note:- Pandas.read_csv( ) & pandas.read_excel( ) can do the transformation to dates when reading the data using the "parse_dates" parameter with a list of the columns to read as Timestamps.

Syntax: Pd.read_csv('file', parse_dates=['date'])

→ to fix warnings we use either : dayfirst=True or date_format.

Working with 'Date Time' in Pandas.

Get year, month & day.

df['year'] = df['date'].dt.year

df['month'] = df['date'].dt.month

df['day'] = df['date'].dt.day

Get week of year, the day of week and leap year.

df['week_of_year'] = df['date'].dt.isocalendar

df['day_of_week'] = df['date'].dt.dayofweek

df['is_leap_year'] = df['date'].dt.is_leap_year

Get the year from the date of birth

today = pd.to_datetime('today')

df['age'] = today.year - df['dob'].dt.year

→ df['dob'].dt.day_name()

month_name().

Pandas Timedelta :- Represents a duration, the difference b/w two dates or times. Many properties can be accessed using .dt like components, days, seconds etc. We can also access timedelta methods using .dt accessor like total_seconds()

eg. df['delivery time'].dt.Components

o/p - days, hours, minutes, milliseconds, microseconds, nanoseconds

.dt.total_seconds()

Improve performance by Setting Date Column as the Index.

Syntax- df = df.set_index(['date col'])

modifying the index inplace

df.set_index('DateCol', inplace = True)

Select data with a specific year and perform aggregation.

select data with a specific year

df.loc['2018']

by specific day

df.loc['2018-5-1']

by using slicing

df.loc['2018-5-1':'2018-5-5']

Applying aggregation within a date slicing.

df.loc['2025-1-1':'2018-5-5']['sales'].mean()

Sorting Data Based on Index or Values & Sorting Index

```
df.sort_index(ascending = False)
df.sort_values(by = 'Index')
df.sort_index()
```

GroupBy: It is a function in pandas to split the data into groups based on some criteria. This function is useful for summarizing data, performing aggregation and transforming data.

→ groupby() and pivot_table() are very powerful in analyzing and summarizing the data. pivot_table() are more powerful when applying complex aggregation operations.

Data

- Qualitative (Categorical)
  - Ordinal [Order]
    - Eg: Grades, Rating
    - [A>B>C>D]
  - Nominal
    - Eg: Gender, Nationality etc
    - [Style put >, <, = it is not happen then it's nominal]
- Quantitative (Numerical)
  - Real (or Continuous)
    - eg: weight, height, Height, Width, H (cm), BMI
  - Discrete
    - Age, Month, Group, Rank

Ordinal — Order

Nominal — no Order

→ Groupby Operation is applied on Categorical col or discrete [with countable values only]

Eg:

	H(cm)	W(kg)	G
0	150	45	F
1	180	90	F
2	190	45	M
3	145	80	M
4	170	90	F

→ df.groupby('G').

F → [0, 1, 4].

M → [2, 3].

→ There are two groups → F & M.

Female group has 3 data Points, Male has 2 data Points.

Dataframe groupby method that splitting of data into various groups

groups — The groups attribute is a dictionary whose keys are the computed unique groups and corresponding values are the axis labels belonging to each group.

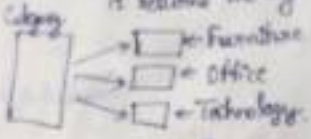
df.groups

Get unique group keys.

grouped_df.groups.keys()

↓

It returns the key values. i.e; group names



grouped_df

grouped_df = df.groupby('Category')

grouped_df.groups

grouped_df.groups.keys()

→ Filter data on the basis of group keys.

grouped_df.get_group('Technology')

↓

to retrieve a group from the grouped object.

Returning first row, last row and nth row for each group.

Grouped_df.head() → It returns the top 5 first rows

grouped.first() --> It will fetch those values the three split first row

grouped.nth(n) --> It returns the three split nth row

Grouping the Data Based on Product Category & Subcategory

#Grouping based on Category first and then subcategory

grouped = df.groupby(['Category', 'subcategory'])

#Filter data on the basis of group keys

grouped.get_group(('Technology', 'Phones'))

↓ Category ↓ Subcategory

→ We can use groups, keys(), .first(), .last(), .nth() in this Category & Subcategory grouping also.

↓ Filtration

Grouping And Aggregation:-

split-apply-combine.

↓

Split :- the data into groups.

apply :- apply a function to each group independently.

Combine :- Combine the results into a data structure.

In apply step, we might wish to do one of the following:

Aggregation :- Compute a summary statistic (or statistics) for each group

Compute group sums or means.
Compute group sizes/counts.

Filtration - True or false.
→ Discard data that belong to groups with only a few members.
→ Filter out data based on the group sum or mean.

Transformation - perform some group-specific computations and return a like-indexed object.
→ Standardize data (zscore) within a group.
→ Filling NAs within groups with a value derived from each group.

Split → Grouping patterns?

Apply → Aggregation, Filtration, Transformation.

→ While we applied filtration & Aggregation combine process is also done.

Built-in Aggregation Methods:
→ any(), all(), count(), cov(), first(), idxmax(), idxmin(), idxset(), max(), mean(), median(), min(), nunique(), prod(), sem(), size(), std(), quantile(), sum(), var().

Built-in Filtration:
head() — select the top row(s) of each group.
nth() — select the nth row(s) of each group.
tail() — select the bottom row(s) of each group.

→ .filter — It will filter the data based upon some condition.

.filter(lambda group:group['Sales'].mean()>300).

Transformation -

Op:

Cat1	Cat2
A	1
B	4
A	4
C	3
B	7
A	5

[0, 1, 2, 3, 4] ---> [0, 1, 2, 3, 4].

Combining Data From Multiple Tables using pd.merge() function.

→ Multiple tables can be combined both column-wise and row-wise using the "concat" function.

→ For database-like merging/joining of tables, use merge function.

Syntax:- pd.merge(left, right, how='inner', on=None)

→ Conforms designs schema for database. ---> § Normalized Tables § ie;

Normalization
- 1NF
- 2NF
- 3NF
- BCNF

Normalization - It means you apply a process of normalizing the entire schema by following the first, 2nd, 3rd & BCNF.

Pd.merge() ---> Joining of multiple different tables.

→ In pandas function merge & join concept in SQL

How argument:-

→ how to merge i.e how to determine which keys are to be included in the resulting table. If a key combination does not appear in either the left or right tables, the values in the joined table will be NA.

how

Inner ---> Display matching records from both tables.

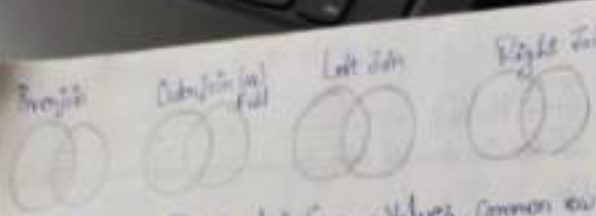
left ---> Display all records from the left table and matching records from the right table.

right ---> Display all records from the right table and matching records from the left table.

Outer ---> keep all the records from both tables.

Cross ---> Cartesian product.

Inner Join Outer Join (or) Full Left Join Right Join



Inner Join → Resultant is (Common Values, Common rows (Values)).

df1

Zooid	Country
101	C4
102	C4
103	C2
104	C3

df2

Pandas name	Zooid
Julie	102
Jeff	105
Lauren	106
Jeff	101
dan	101
mack	103

pd.merge(df1, df2, On = "Zooid", how = "Inner")

↓

Common Column to fix two dfs.

o/p:-

Zooid	Country	Pandas name
101	C4	Jeff
101	C4	dan
102	C4	Julie
103	C2	mack

→ Inner is "default".

Full Outer Join:-

→ This keeps all the rows from both Dataframes, filling in NaNs, where there are no matches. It is similar to a full outer join in SQL.

pd.merge(df1, df2, On = "Zooid", how = "Outer")

o/p:-

Zooid	Country	Pandas name
101	C4	Jeff
101	C4	dan
102	C4	Julie
103	C2	mack
104	C3	NaN
105	NaN	Jeff
106	NaN	Lauren

Left Join:- Very focused on rows in the left table

df1 → left dataframe

df2 → right dataframe.

pd.merge(df1, df2, On = "Zooid", how = "left")

Resultant:- This keeps all rows from the left DataFrame (df1) and only the matching rows from the right DataFrame (df2). If there is no match, NaN values are placed in the resulting DF for missing columns from 'df2'.

O/p:-

Zone_id	country	parcels
102	CY	Will
102	CY	alan
102	CY	Julie
103	CZ	mark
104	CZ	Nikki

Right Join:- Very focused on right table.

pd.merge(df2, df1, on="Zone_id", how="right").

Resultant:- This keeps all rows from the right df (df2), and only the matching rows from the right DataFrame (df1). If there is no match, NaN values are placed in the resulting DF for missing columns from 'df1'.

O/p:-

Zone_id	Country	Parcels
102	CY	Julie
105	Nan	Jot
105	Nan	warn
101	CY	bill
101	CY	alan
103	CZ	mark

→ 1. One-to-one joins:- for example when joining two DataFrames objects on their indexes (which must contain unique values).

→ 2. Many to one:- for example when joining an index (unique) to one or more columns in a different DataFrame.

→ 3. Many to Many:- joining columns on columns.

Pivot table:

→ Pivot table are powerful tool for data analysis and summarization. They allow you to transform & aggregate data in a way that is more easy to understand & analyze.

To create a pivot table: Use the 'pivot_table' function to create a pivot table.

Visual data Analysis Using Pandas (Plotting)

Syntax:- DataFrame.plot(kind='kind')

→ line : line plot (default)
bar : vertical bar plot
barh : horizontal bar plot
hist : histogram
box : boxplot
kde : Kernel Density Estimation plot
density :- same as 'kde'
area : area plot
scatter : scatter plot
pie : pie plot
hexbin : hexbin plot.

→ Outliers:- Any extremely large to extremely small value, when compared to ~~rest of the data (when compared rest~~ of the data.

Mean, Median, Mode [Categorical]

Univariate Analysis.

Univariate

1 Variable or Column Feature

→ min, max, Count, nunique → can also run visual Univariate Analysis.

Non Visual Analysis

Descriptive Stats & Basics of Probability

- Univariate
  - (count()
  - unique()
  - (value_counts()
  - Categorical
  - value()
  - max()
  - min()
  - count()
  - mean()
  - median()
  - mode() — Numerical
  - Value() — Categorical
  - Std() — Not to mean
  - Var()
  - Mean() — Num.
  - Range()
  - Table() — not to mention
- Bivariate
  - corr()
  - Covar()
  - (Correlation w.r.t)
  - 1. Strength
    - to 1 — very strong
    - 0 no relationship
- Only Categorical data
  - Univariate
    - Maximum probability
  - Bivariate
    - (MAX)
    - (MIN)
    - (MAX)

Visualization:-

EDA → Non-Viz Analysis (Univariate & Bivariate)
Viz Analysis (Univariate & Bivariate)

Visual Analysis:-

1. Univariate Analysis

	Where
• Bar Plot/Count Plot	→ Categorical data, discrete Numerical data.
• Histogram plot	→ Real Numerical (MAX) or Continuous Numerical
• KDE/PDF plot	→ Real/Continuous Numerical.
• Box Plot	→ Real/Continuous Numerical.

What to infer?

• Bar Plot/Count plot	→ Distribution & Outliers.
• Histogram plot	→ } Distribution of data.
• KDE/PDF plot	→ }
• Box Plot	→ to detect Outliers

Purpose of Univariate Analysis (Visual)

1. To study the Distributions
2. To detect the Outliers

2. Bar/Count plot → on Cat/Discrete Num → ① & ②
3. Histogram plot → on Real/Continuous Num → ①
4. CDF/PDF plot → Real/Continuous Num → ①
5. Box plot → Real/Continuous Num → ②

2. Bivariate Analysis

Purpose → to detect the Relationships

I. Num vs Num → Scatter Plot
II. Cat vs Num → Box plot.
III. Cat vs Cat → Stacked Bar/Count plot.
IV. Date/Time vs num → Line plot
(More Series Plots)

→ Visual Analysis much more reliable than compared to non-Visual Analysis.
We can find the Outliers based upon on the Value Counts in Non Visual Analysis

Bar Plot/Count Plot:-

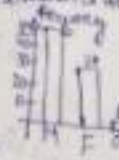
Univariate → Categorical or Discrete Num
Bivariate → Cat vs Cat col [Called as stacked bar plot].

Non-Visual technique → Distribution of Gender Col

df[G].value_counts() → always normalize this value

M 50
F 30

Visual technique → Distribution of Gender Col? (X, Y)



→ A data point which is very less frequent that is the Outlier datapoint.

Histogram Plot:-

[155, 160, 161, 160.5, 170, 165, 162, 153, 152.5, 170, 173, 173, 175, 178, 181]

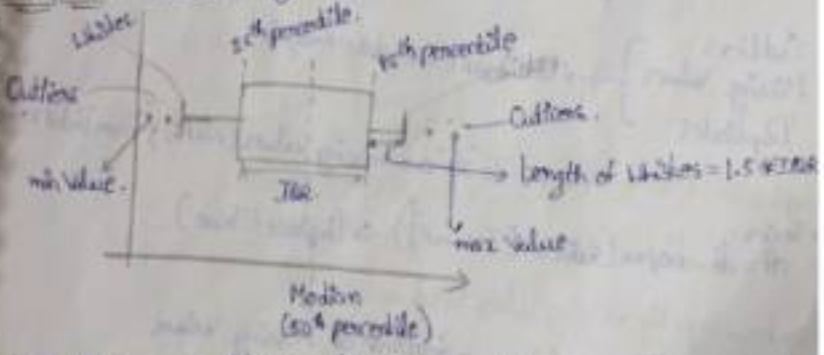
On Real/Continuous Numbers.

155, 155.5 → 150-155
159, 160, 162 → 155-160
160.5, 161 → 160-165 → 165-170
173, 173, 175, 175 → 170-175
178, 180 → 175-180
181, 182. → 180-185.

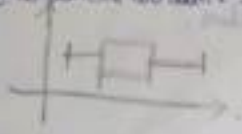
Same size of bins.
It is known as Bucketing/Binning
Size = 5 units.

There is no space b/w the bars, if spaces will exist if there is no data in the region.

Box Plot:- It is a simply looks like a box.



→ If there are no outliers we didn't see any dots.



Graph Data Viz

→ Convert to large the value of the data, changing of multiple are needed
→ to convert the most to, to data is received from google sheet

Import data
data.read("[illegible]")
→ 2 line → [illegible] of data & [illegible]
df.head
df.shape

Clean

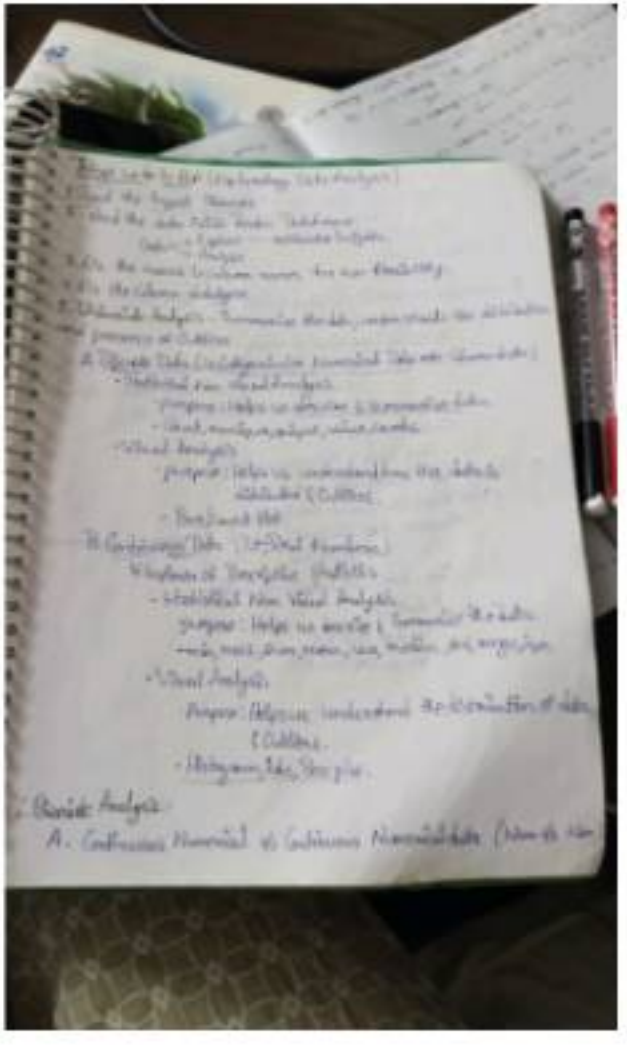
→ Dropping [illegible] — [illegible] would be [illegible] or not you have to check.
df.dropna — [illegible] drop) then [illegible] to df.dropna()
[illegible] df[illegible]) by using this you can reflect your [illegible] [illegible]
True

→ Syntax — [illegible] dropping
df.drop( )

Outliers
Missing values
Duplicates
→ [illegible] Analysis

Dropping of Missing values → to remove missing values (NaN) from [illegible]
→ Syntax
df.dropna(axis=0, inplace=True)
→ axis=0 (drop rows) → inplace=True

For handling the missing values in column wise
df.fillna( ) → [illegible] all missing values
→ Replace [illegible] missing values



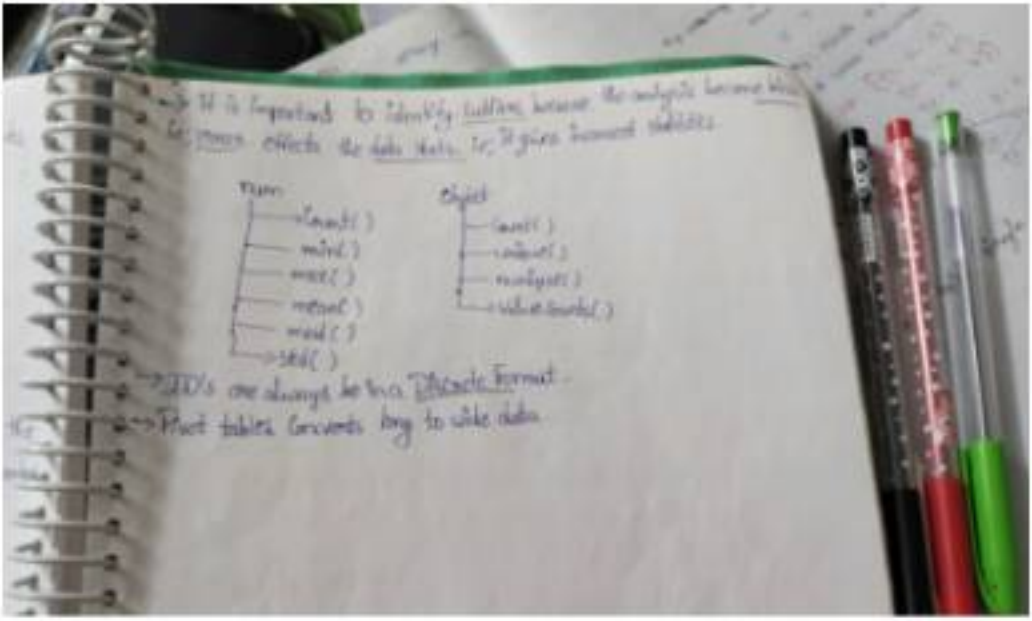
- Statistical Non Visual Analysis
  - purpose - Is there any relationship b/w two variables
    Pearson or non Pearson Correlation
  - Pearson Correlation Coefficient
- Visual Analysis
  - Scatter Plot

B. Continuous numerical vs Discrete Data (Cat vs Num)

(groupby ( )
- Statistical Non Visual Analysis
  - purpose - How many discrete groups are there & does the individuals in the groups independent & dependent
  - Compare the Mean, Median, std of the groups
- Visual Analysis
  - Box plots & Histogram plots

C. Discrete vs Discrete Data (Cat vs Cat)

- Statistical Non Visual Analysis
  - Purpose - Are the individuals in the groups independent or dependent?
  - Cross tabs (or) Frequency Tables
- Visual Analysis
  - Stacked Bar Plot
  - Unstacked Bar Plot


It is important to identify outliers because the analysis becomes biased
Outliers affects the data sets. So, it gives incorrect statistics
num
count( )
min( )
max( )
mean( )
mod( )
std( )
object
count( )
unique( )
nunique( )
value_counts( )
DD's are always be in a Dataset format.
Pivot tables converts long to wide data